Project APEX Executable Lab Manual

Twenty guided experiments from units to latent planning

OpenAI

August 2026

Table of Contents

1 How to run a lab

2 Lab 1: Units and tensor axes

2.1 Why this lab exists

2.2 Predict before running

2.3 Execute

1 How to run a lab

For every lab, use the same discipline:

Read the goal and predict the output.

Copy the solution into a temporary file by hand or complete your own starter.

Run it once unchanged.

Explain every printed number and tensor axis.

Introduce the specified defect deliberately.

Find the earliest contract that fails.

Repair it and add a regression assertion.

Complete the extension without copying.

Record how the idea changes Project APEX.

The lab is complete only when you can alter it and still explain the result.

2 Lab 1: Units and tensor axes

2.1 Why this lab exists

A number without a unit and an array without axis names are both incomplete evidence. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

2.2 Predict before running

Predict the converted speeds and the exact [time, feature] shape before running. Write the expected output, shapes, units and likely failure modes.

2.3 Execute

```
cd labs/01_units_and_shapes
python solution.py
```

2.4 Complete code with line numbers

```
1: import numpy as np
2:
3: speed_kmh = np.array([0.0, 72.0, 144.0])
4: speed_mps = speed_kmh / 3.6
5: telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
6: print("speed_mps:", speed_mps)
7: print("telemetry shape [time, features]:", telemetry.shape)
8: assert telemetry.shape == (3, 2)
```

2.5 Packaged reference output

```
speed_mps: [ 0. 20. 40.]
telemetry shape [time, features]: (3, 2)
```

2.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

2.7 Break it deliberately

Transpose the array or skip km/h conversion. Add assertions for units, shape and feature order.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

2.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

2.9 Extension

Add a typed telemetry batch carrying values, names, units and sample rate.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

2.10 Where it enters APEX

Canonical adapter and contract validation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

2.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

3 Lab 2: Differentiate and integrate a speed trace

3.1 Why this lab exists

Numerical derivatives magnify noise; numerical integration accumulates bias. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

3.2 Predict before running

Predict whether reconstructing speed from a numerical gradient will be exact. Write the expected output, shapes, units and likely failure modes.

3.3 Execute

```
cd labs/02_vectorized_kinematics
python solution.py
```

3.4 Complete code with line numbers

```
1: import numpy as np
2:
3: dt = 0.2
4: time = np.arange(0, 5, dt)
5: speed = 20 + 4*np.sin(time)
6: accel = np.gradient(speed, dt)
7: reconstructed = speed[0] + np.cumsum(accel)*dt
8: print("first accelerations:", accel[:5].round(3))
9: print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

3.5 Packaged reference output

first accelerations: [3.973 3.894 3.66 3.279 2.768]
max reconstruction error: 0.7946773231802453

3.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

3.7 Break it deliberately

Change dt without changing time samples; compare errors and locate the contract mismatch.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

3.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

3.9 Extension

Compare forward, central and smoothed derivatives.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

3.10 Where it enters APEX

Synthetic causal generator and physical sanity checks. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

3.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

4 Lab 3: Longitudinal forces

4.1 Why this lab exists

Separating drive, brake and drag makes causal errors visible. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

4.2 Predict before running

Calculate net force and acceleration for one chosen row by hand. Write the expected output, shapes, units and likely failure modes.

4.3 Execute

```
cd labs/03_force_model
python solution.py
```

4.4 Complete code with line numbers

```
1: from education.core import CarState, step_car
2:
3: state = CarState(0.0, 0.0, 40.0)
4: for i in range(5):
5:     state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)
6:     print(i, state)
```

4.5 Packaged reference output

```
Traceback (most recent call last):
  File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/03_force_model/solution.py", line 1, in
<module>
    from education.core import CarState, step_car
ModuleNotFoundError: No module named 'education'
```

4.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

4.7 Break it deliberately

Feed km/h into a coefficient calibrated for m/s and add a property test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

4.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

4.9 Extension

Add grip-limited traction and show where more throttle stops helping.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

4.10 Where it enters APEX

Synthetic telemetry dynamics and residual modelling. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

4.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

5 Lab 4: Sampling and aliasing

5.1 Why this lab exists

The model cannot learn temporal information the sensor never captured. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

5.2 Predict before running

Predict the apparent signal at 5, 12 and 40 Hz. Write the expected output, shapes, units and likely failure modes.

5.3 Execute

```
cd labs/04_sampling_aliasing
python solution.py
```

5.4 Complete code with line numbers

```
 1: import numpy as np
 2:
 3: for hz in [5, 12, 40]:
 4:     t=np.arange(0,2,1/hz)
 5:     signal=np.sin(2*np.pi*8*t)
 6:     print(hz, "Hz samples:", np.round(signal[:8],3))
```

5.5 Packaged reference output

```
5 Hz samples: [ 0.    -0.588  0.951 -0.951  0.588 -0.    -0.588  0.951]
12 Hz samples: [ 0.    -0.866  0.866 -0.    -0.866  0.866 -0.    -0.866]
40 Hz samples: [ 0.     0.951  0.588 -0.588 -0.951 -0.     0.951  0.588]
```

5.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

5.7 Break it deliberately

Choose a signal above Nyquist and show two different continuous signals sharing samples.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

5.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

5.9 Extension

Build an anti-alias filter experiment.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

5.10 Where it enters APEX

Canonical frequency and F1 25 telemetry-rate decisions. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

5.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

6 Lab 5: Ridge transition baseline

6.1 Why this lab exists

A simple control tells you whether complexity earns its cost. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

6.2 Predict before running

Predict coefficient signs for speed, throttle and brake. Write the expected output, shapes, units and likely failure modes.

6.3 Execute

```
cd labs/05_regression_baseline
python solution.py
```

6.4 Complete code with line numbers

```
1: import numpy as np
2: from sklearn.linear_model import Ridge
3:
4: rng=np.random.default_rng(4)
5: speed=rng.uniform(20,80,1000)
6: throttle=rng.uniform(0,1,1000)
```

```
 7: brake=rng.uniform(0,1,1000)
 8: y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
 9: X=np.column_stack([speed,throttle,brake])
10: model=Ridge(alpha=1.0).fit(X[:800],y[:800])
11: print("coefficients:", model.coef_)
12: print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))
```

6.5 Packaged reference output

```
coefficients: [ 0.91911817  0.68984381 -1.03384695]
test MAE: 0.18453407932904015
```

6.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

6.7 Break it deliberately

Leak the target into features, observe near-perfect error, then prevent it with an allow-list.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

6.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

6.9 Extension

Add polynomial speed features and inspect residuals.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

6.10 Where it enters APEX

Baseline-relative model evaluation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

6.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

7 Lab 6: Learning a physical coefficient

7.1 Why this lab exists

Autograd is easiest to trust when the expected parameter is known. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

7.2 Predict before running

Predict the sign of the drag-coefficient gradient from an under-estimate. Write the expected output, shapes, units and likely failure modes.

7.3 Execute

```
cd labs/06_autograd_drag
python solution.py
```

7.4 Complete code with line numbers

```
 1: import torch
 2:
 3: speed=torch.linspace(5,70,200)
 4: true_drag=0.002
 5: target=-true_drag*speed**2
 6: drag=torch.tensor(0.01,requires_grad=True)
 7: opt=torch.optim.SGD([drag],lr=1e-7)
 8: for step in range(200):
 9:     opt.zero_grad()
10:     pred=-drag*speed**2
11:     loss=((pred-target)**2).mean()
12:     loss.backward(); opt.step()
13: print("learned drag:", drag.item(), "loss:", loss.item())
```

7.5 Packaged reference output

learned drag: 0.0020000000949949026 loss: 0.0

7.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

7.7 Break it deliberately

Omit zero_grad and increase learning rate until divergence.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

7.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

7.9 Extension

Estimate two correlated force parameters and study identifiability.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

7.10 Where it enters APEX

Training-loop diagnostics and hybrid physics learning. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

7.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

8 Lab 7: A transition MLP

8.1 Why this lab exists

A module is a parameterized function with a precise shape contract. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

8.2 Predict before running

Write input/output shapes for one batch before executing. Write the expected output, shapes, units and likely failure modes.

8.3 Execute

```
cd labs/07_pytorch_module
python solution.py
```

8.4 Complete code with line numbers

```
 1: import torch
 2: from torch import nn
 3:
 4: class Transition(nn.Module):
 5:     def __init__(self):
 6:         super().__init__()
 7:         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
 8:     def forward(self,x): return self.net(x)
 9:
10: model=Transition(); x=torch.randn(4,5); y=model(x)
11: print(model); print("input",x.shape,"output",y.shape)
```

8.5 Packaged reference output

```
Transition(
  (net): Sequential(
    (0): Linear(in_features=5, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=2, bias=True)
  )
)
input torch.Size([4, 5]) output torch.Size([4, 2])
```

8.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

8.7 Break it deliberately

Swap batch and feature axes; block the bug with exact assertions.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

8.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

8.9 Extension

Add a residual connection predicting state delta instead of absolute state.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

8.10 Where it enters APEX

Frame transition baseline and shape tracing. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

8.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

9 Lab 8: Causal history and target windows

9.1 Why this lab exists

Sequence slicing defines what the model is allowed to know. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

9.2 Predict before running

List raw indices for the first and final example. Write the expected output, shapes, units and likely failure modes.

9.3 Execute

```
cd labs/08_dataset_windows
python solution.py
```

9.4 Complete code with line numbers

```
 1: import numpy as np
 2: from education.core import sliding_windows
 3:
 4: values=np.arange(30,dtype=float).reshape(10,3)
 5: x,y=sliding_windows(values,history=4,horizon=2)
 6: print("x",x.shape,"y",y.shape)
 7: print("first history\n",x[0]); print("first future\n",y[0])
```

9.5 Packaged reference output

```
Traceback (most recent call last):
  File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/08_dataset_windows/solution.py", line 2, in <module>
    from education.core import sliding_windows
ModuleNotFoundError: No module named 'education'
```

9.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

9.7 Break it deliberately

Put target state into future inputs and add a leakage test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

9.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

9.9 Extension

Add future controls as a separate tensor and session identifiers.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

9.10 Where it enters APEX

APEX WindowDataset and rollout contract. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

9.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

10 Lab 9: Loss functions under outliers

10.1 Why this lab exists

Loss determines which errors receive optimization pressure. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

10.2 Predict before running

Rank MSE, MAE and Huber sensitivity to one extreme residual. Write the expected output, shapes, units and likely failure modes.

10.3 Execute

```
cd labs/09_loss_comparison
python solution.py
```

10.4 Complete code with line numbers

```
1: import torch
2: from torch.nn import functional as F
3:
4: pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
5: print("MSE",F.mse_loss(pred,target).item())
6: print("MAE",F.l1_loss(pred,target).item())
7: print("Huber",F.huber_loss(pred,target).item())
```

10.5 Packaged reference output

```
MSE 72.25
MAE 4.25
Huber 4.125
```

10.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

10.7 Break it deliberately

Scale one feature by 100 without normalization and observe domination.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

10.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

10.9 Extension

Create a weighted multi-feature telemetry loss.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

10.10 Where it enters APEX

Model training versus operational metrics. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

10.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

11 Lab 10: Gradients and updates

11.1 Why this lab exists

Backward computes gradients; the optimizer changes parameters. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

11.2 Predict before running

State what changes after each training-loop line. Write the expected output, shapes, units and likely failure modes.

11.3 Execute

```
cd labs/10_training_loop
python solution.py
```

11.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
5: model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
```

```
 6: for epoch in range(8):
 7:     model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
 8:     grad=float(model.weight.grad.norm()); opt.step()
 9:     model.eval();
10:      with torch.no_grad(): val=((model(x)-y)**2).mean()
11:      print(epoch,float(loss),grad,float(val))
```

11.5 Packaged reference output

```
0 3.2264039516448975 3.7724902629852295 2.9054489135742188
1 2.9054489135742188 3.5811667442321777 2.6067750453948975
2 2.6067750453948975 3.390789270401001 2.3303141593933105
3 2.3303141593933105 3.2017178535461426 2.0752670764923096
4 2.0752670764923096 3.0143868923187256 1.8400570154190063
5 1.8400570154190063 2.829359292984009 1.6230223178863525
6 1.6230223178863525 2.6473701000213623 1.4232583045959473
7 1.4232583045959473 2.469295024871826 1.2406574487686157
/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/10_training_loop/solution.py:11:
UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected
behavior.
Consider using tensor.detach() first. (Triggered internally at
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
  print(epoch,float(loss),grad,float(val))
```

11.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

11.7 Break it deliberately

Remove zeroing, omit eval mode and create broadcasting targets.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

11.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

11.9 Extension

Write a JSON trace of gradients and update norms.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

11.10 Where it enters APEX

APEX training and checkpoint validation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

11.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

12 Lab 11: Recurrent state updates

12.1 Why this lab exists

A recurrent model is a transition over hidden memory. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

12.2 Predict before running

Compute the first scalar hidden update by hand. Write the expected output, shapes, units and likely failure modes.

12.3 Execute

```
cd labs/11_rnn_from_scratch
python solution.py
```

12.4 Complete code with line numbers

```
1: import torch
2:
3: torch.manual_seed(0); W=torch.randn(3,2); U=torch.randn(3,3); b=torch.zeros(3); h=torch.zeros(3)
4: sequence=torch.tensor([[1.,0.],[0.,1.],[1.,1.]])
```

```
5: for t,x in enumerate(sequence):
6:     h=torch.tanh(W@x+U@h+b); print(t,h.numpy().round(3))
```

12.5 Packaged reference output

```
0 [ 0.912 -0.975 -0.795]
1 [-0.169  0.563 -0.984]
2 [ 0.982 -0.968 -0.582]
```

12.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

12.7 Break it deliberately

Reverse sequence order and reuse state across sequences.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

12.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

12.9 Extension

Create a long-delay memory task.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

12.10 Where it enters APEX

Why APEX needs history encoding. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

12.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

13 Lab 12: GRU memory gates

13.1 Why this lab exists

Gates learn how much old memory to retain or overwrite. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

13.2 Predict before running

Predict whether repeating the same input yields identical hidden states. Write the expected output, shapes, units and likely failure modes.

13.3 Execute

```
cd labs/12_gru_gates
python solution.py
```

13.4 Complete code with line numbers

```
 1: import torch
 2:
 3: torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
 4: for x in [torch.tensor([[1.,0.]]),torch.tensor([[0.,1.]]),torch.tensor([[0.,1.]])]:
 5:     h=cell(x,h); print(h.detach().numpy().round(3))
```

13.5 Packaged reference output

```
[[ 0.354 -0.151 -0.384  0.149]]
[[ 0.278  0.282 -0.371  0.408]]
[[ 0.322  0.39  -0.39   0.542]]
```

13.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

13.7 Break it deliberately

Force update gates to saturation and inspect memory.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

13.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

13.9 Extension

Compare GRU and matched RNN on delayed events.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

13.10 Where it enters APEX

Primary deterministic world model. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

13.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

14 Lab 13: Linear state-space memory

14.1 Why this lab exists

Eigenvalues determine decay, persistence, oscillation and explosion. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

14.2 Predict before running

Predict the two hidden components after the first pulse. Write the expected output, shapes, units and likely failure modes.

14.3 Execute

```
cd labs/13_linear_ssm
python solution.py
```

14.4 Complete code with line numbers

```
 1: import numpy as np
 2: A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
 3: for u in [1,1,0,0,0]:
 4:     x=A@x+B[:,0]*u; print(x.round(3))
```

14.5 Packaged reference output

```
[0.2  0.05]
[0.39  0.112]
[0.37  0.135]
[0.352 0.151]
[0.334 0.164]
```

14.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

14.7 Break it deliberately

Set spectral radius above one and create a stability test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

14.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

14.9 Extension

Design fast, medium and slow memory modes.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

14.10 Where it enters APEX

SSM challenger and hidden-norm monitoring. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

14.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

15 Lab 14: Input-dependent state-space memory

15.1 Why this lab exists

Selection lets current content alter retention and writing. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

15.2 Predict before running

Predict how basis-vector order changes final memory. Write the expected output, shapes, units and likely failure modes.

15.3 Execute

```
cd labs/14_selective_ssm
python solution.py
```

15.4 Complete code with line numbers

```
 1: import torch
 2: from torch import nn
 3:
 4: class SelectiveCell(nn.Module):
 5:     def __init__(self,d):
 6:         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
 7:     def forward(self,x,h):
 8:         a=torch.sigmoid(self.decay(x))*0.99
 9:         b=torch.tanh(self.write(x))
10:         return a*h+(1-a)*b
11: cell=SelectiveCell(3); h=torch.zeros(1,3)
12: for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))
```

15.5 Packaged reference output

```
[[-0.169 -0.022  0.151]]
[[ 0.072  0.103 -0.15 ]]
[[ 0.048  0.153 -0.225]]
```

15.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

15.7 Break it deliberately

Remove gate bounds and inspect explosion/frozen memory.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

15.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

15.9 Extension

Log retention by braking/straight regimes.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

15.10 Where it enters APEX

Selective SSM challenger and Mamba preparation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

15.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

16 Lab 15: Latent compression

16.1 Why this lab exists

Reconstruction pressure does not guarantee predictive representations. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

16.2 Predict before running

Predict latent and reconstruction shapes. Write the expected output, shapes, units and likely failure modes.

16.3 Execute

```
cd labs/15_autoencoder
python solution.py
```

16.4 Complete code with line numbers

```
 1: import torch
 2: from torch import nn
 3:
 4: torch.manual_seed(3); x=torch.randn(256,12)
 5: enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
```

```
 6: for _ in range(80):
 7:     opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward();
opt.step()
 8: print("latent",z.shape,"loss",float(loss))
```

16.5 Packaged reference output

```
latent torch.Size([256, 3]) loss 0.7505914568901062
/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/15_autoencoder/solution.py:8: UserWarning:
Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
  print("latent",z.shape,"loss",float(loss))
```

16.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

16.7 Break it deliberately

Add high-variance nuisance features and observe what latent preserves.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

16.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

16.9 Extension

Add a future-state probe and predictive auxiliary loss.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

16.10 Where it enters APEX

Representation learning before RSSM/JEPA. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

16.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

17 Lab 16: Stochastic latent variables

17.1 Why this lab exists

Reparameterization separates noise from differentiable distribution parameters. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

17.2 Predict before running

Compute standard deviations and KL by hand. Write the expected output, shapes, units and likely failure modes.

17.3 Execute

```
cd labs/16_vae_reparameterization
python solution.py
```

17.4 Complete code with line numbers

```
1: import torch
2:
3: mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[-1.0,0.5]])
4: std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
5: kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
6: print("mu",mu,"std",std,"sample",z,"KL",kl)
```

17.5 Packaged reference output

mu tensor([[0.2000, -0.4000]]) std tensor([[0.6065, 1.2840]]) sample tensor([[0.8058, 0.3872]]) KL tensor([0.3583])

17.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

17.7 Break it deliberately

Set extreme log variance and inspect numerical behaviour.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

17.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

17.9 Extension

Sample repeatedly and compare empirical moments.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

17.10 Where it enters APEX

RSSM posterior/prior distributions. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

17.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

18 Lab 17: Prior and posterior belief

18.1 Why this lab exists

Training may use observation evidence; imagination may not. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

18.2 Predict before running

Name the inputs to h, prior and posterior before running. Write the expected output, shapes, units and likely failure modes.

18.3 Execute

```
cd labs/17_rssm_step
python solution.py
```

18.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
5: h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
6: h=cell(torch.cat([prev_z,action],-1),h)
7: prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
8: print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

18.5 Packaged reference output

```
h torch.Size([2, 8]) prior torch.Size([2, 4]) posterior torch.Size([2, 4])
```

18.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

18.7 Break it deliberately

Leak observation into the prior and write a no-future-observation test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

18.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

18.9 Extension

Add sampling, KL and decoder loss for multiple steps.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

18.10 Where it enters APEX

Dreamer-style latent dynamics. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

18.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

19 Lab 18: Cross-entropy planning

19.1 Why this lab exists

Planning repeatedly samples, imagines, keeps elites and refits. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

19.2 Predict before running

Predict the direction of the first action mean. Write the expected output, shapes, units and likely failure modes.

19.3 Execute

```
cd labs/18_cem_planning
python solution.py
```

19.4 Complete code with line numbers

```
 1: import numpy as np
 2:
 3: rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
 4: for it in range(5):
 5:     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
 6:     speed=40+np.cumsum(2*actions-0.4,axis=1)
 7:     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
 8:     elite=actions[np.argsort(score)[-50:]]
 9:     mean,std=elite.mean(0),elite.std(0)+1e-3
10:     print(it,mean.round(2),score.max().round(3))
```

19.5 Packaged reference output

```
0 [0.71 0.68 0.7  0.62 0.67 0.71] -1.123
1 [0.84 0.8  0.82 0.82 0.83 0.82] -0.019
2 [0.86 0.84 0.86 0.88 0.87 0.9 ] -0.005
3 [0.87 0.85 0.85 0.87 0.89 0.87] -0.001
4 [0.86 0.84 0.85 0.88 0.89 0.87] -0.001
```

19.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

19.7 Break it deliberately

Remove constraints and create a model loophole for the planner to exploit.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

19.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

19.9 Extension

Add brake actions, uncertainty penalty and receding horizon.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

19.10 Where it enters APEX

Scenario planning and later control. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

19.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

20 Lab 19: Causal asynchronous joins

20.1 Why this lab exists

Alignment policy determines whether future information leaks backward. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

20.2 Predict before running

Manually match every car timestamp under backward tolerance. Write the expected output, shapes, units and likely failure modes.

20.3 Execute

```
cd labs/19_time_alignment
python solution.py
```

20.4 Complete code with line numbers

```
 1: import pandas as pd
 2:
 3: car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
 4: weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
 5:
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=
```

```
0.3)
  6: print(aligned)
```

20.5 Packaged reference output

```
t  speed  rain
0  0.0    10   NaN
1  0.2    12   0.0
2  0.4    14   NaN
3  0.6    16   0.3
```

20.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

20.7 Break it deliberately

Switch to nearest and add a future spike.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

20.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

20.9 Extension

Persist source age and missingness masks.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

20.10 Where it enters APEX

FastF1/OpenF1 stream alignment. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

20.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

21 Lab 20: Staged artifact workflow

21.1 Why this lab exists

Durable boundaries make retries, lineage and debugging possible. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

21.2 Predict before running

Predict the exact output tree. Write the expected output, shapes, units and likely failure modes.

21.3 Execute

```
cd labs/20_pipeline_contract
python solution.py
```

21.4 Complete code with line numbers

```
1: from pathlib import Path
2: import json
3:
4: run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
5: stages=['ingest','validate','window','train','evaluate','publish']
6: for i,name in enumerate(stages):
7:     path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
8:     print(path)
```

21.5 Packaged reference output

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

21.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

21.7 Break it deliberately

Overwrite one shared latest path and simulate failure halfway.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

21.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

21.9 Extension

Add content hashes, atomic writes and resumable status.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

21.10 Where it enters APEX

Local runner, Airflow DAG and registry. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

21.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?